PateLink

Unified Control Paradigm for Embedded Device Control

Joni Kemppainen (1,2), Jori-Aleksi Kemppainen (2)

(1) DEV Joni, Kukkulantie 4, 89400 Hyrynsalmi, Kainuu, Finland
(2) Alfrisin Oy, Tiilitörmäntie 6, 89400 Hyrysalmi, Kainuu, Finland

Contact: solutions@devjoni.com

1st of May 2026

Abstract

Computer-controlled embedded device management remains fragmented due to the absence of a unified control standard. Across industrial automation, scientific instrumentation, and embedded systems integration, this fragmentation increases implementation complexity, limits interoperability, and enforces vendor lock-in. Existing approaches prioritize transport-layer connectivity, often leaving command semantics inconsistent across devices and platforms. PateLink defines a vendor-independent control paradigm to address this gap through two architectural components: (1) a semantic, key-value communication protocol for standardized discovery and command execution, and (2) a hierarchical programming interface that unifies low-level control with high-level software abstraction. By decoupling transport from semantics, PateLink shifts interoperability from connection compatibility to shared operational meaning. This paradigm establishes a modular framework for plug-and-play control across heterogeneous hardware environments. It reduces integration overhead, improves system portability, and provides a consistent foundation for the deployment, maintenance, and scaling of automated systems. PateLink is proposed as a general architecture for embedded interoperability, rather than a transport-specific implementation.

Keywords: Communication protocol, API, programming interface, semantic, unified, fragmentation, automation, computer control, embedded, robotics

Author Transparency

Joni Kemppainen is the principal architect of the PateLink framework. His work focuses on foundational technologies for open-science instrumentation, primarily through his consultancy, **DEV Joni**. He also serves as the Chief Technology Officer at **Alfrisin Oy**, where he leads core product development.

Jori-Aleksi Kemppainen is the Chief Executive Officer of Alfrisin Oy, a provider of modular automation solutions for industrial and scientific environments. Alfrisin Oy is the first commercial entity to implement the PateLink paradigm within its product ecosystem.

1. Fragmentation Problem

While embedded device control appears standardized due to a broad range of physical connectors and communication protocols, this abundance often creates a false sense of interoperability. In practice, the ecosystem remains deeply fragmented because standardization at the **transport layer (the syntactic layer)** rarely extends to the **semantic layer** - the layer defining command structure, device behavior, and control logic.

This creates a systemic gap between data transfer and data meaning. While existing standards specify how data is transmitted, they rarely define a vendor-independent method for interpreting that data. This gap is the root cause of interoperability challenges across industrial automation, scientific instrumentation, and robotics.

1.1. Transport is not logic

Most conventional communication standards are transport-layer focused. Protocols such as SPI, I2C, UART, and CAN define signaling, timing, and packet transmission, but they do not prescribe what the transmitted information represents.

Consequently, manufacturers must overlay proprietary command structures, register maps, and control schemas onto these transport layers. Two devices may share the same physical bus and signaling protocol yet remain functionally incompatible because their command semantics differ. For developers and system integrators, communication standardization does not equate to functional interoperability. Each new device requires:

- Manual interpretation of datasheets.
- Vendor-specific register mapping.
- Custom firmware and software integration.

In effect, the transport layer solves connectivity, but not control.

1.2. Technical Impact: Excessive Integration Burden

Fragmentation imposes a significant "integration tax" on developers and researchers. In multi-device systems - such as automated laboratories or robotics platforms - users encounter three primary barriers:

1.2.1. SDK Proliferation

Manufacturers typically provide proprietary SDKs or driver stacks. For the developers, integrating multiple devices creates a massive technical overhead; a system of ten devices may require ten separate software libraries, ten distinct documentation ecosystems, and ten different programming models. This prevents modularity and forces developers to manage a fragmented landscape of incompatible workflows.

1.2.2. API Incompatibility

Vendor SDKs often impose conflicting architectural assumptions, such as:

- Polling vs. event-driven models.
- Blocking vs. asynchronous APIs.
- OS-specific dependencies and runtime requirements

These conflicts often necessitate complex workarounds, such as virtualization or custom middleware.

1.2.3. Reduced Portability

Because software logic is tightly coupled to vendor-specific interfaces, replacing a single component often requires substantial code redevelopment. This reduces system modularity and prevents the transfer of automation workflows across different hardware environments.

1.3. Economic impact: Wasted resources

Fragmentation is a structural inefficiency that diverts engineering effort from innovation toward repetitive integration work. This burden manifests in two ways:

- **Development Costs:** Engineering teams spend disproportionate capital bridging incompatible interfaces rather than developing higher-value system capabilities.
- **Vendor Lock-in:** When replacing a device requires rewriting the integration logic, organizations are economically discouraged from adopting superior or more cost-effective alternatives.

Furthermore, the long-term maintenance of fragmented stacks creates a continuous operational burden. As vendor libraries are updated or deprecated, organizations must invest sustained engineering effort simply to preserve existing functionality, reducing overall organizational agility.

1.4. Scope and boundaries

This fragmentation is not universally detrimental. Specialized applications - such as high-bandwidth imaging or ultra-low-latency sensor fusion - require optimized, domain-specific protocols that prioritize throughput over abstraction. However, for the majority of embedded control applications in industrial, scientific, and robotic sectors, the primary requirement is not maximal bandwidth, but reliable, portable, and standardized control.

1.5. Conclusion

The modern embedded ecosystem possesses abundant transport standards but lacks a sufficiently universal control paradigm. Existing solutions either address only portions of the stack or remain too specialized, heavyweight, or vendor-constrained to solve fragmentation comprehensively.

There is therefore a critical need for a unified, lightweight, vendor-independent framework that bridges hardware communication, firmware abstraction, and host software integration into a cohesive system. By addressing the gap between transport and meaning, such a paradigm can substantially reduce integration complexity while improving interoperability, portability, and long-term scalability.

2. The Semantic Communication Protocol

PateLink is an architectural framework that decouples hardware implementation from software control through standardized semantic abstraction. It is built upon two foundational pillars: (1) a semantic communication protocol and (2) a unified programming interface.

Rather than layering new abstractions atop fragmented legacy assumptions, PateLink is a clean-room framework designed to unify semantic device control from the outset. The communication architecture establishes how devices describe themselves, expose capabilities, and maintain interoperability, independent of vendor-specific register maps or proprietary conventions.

2.1. The Communication Protocol: Semantic Device Abstraction

Traditional embedded systems are register-centric. To control a device, a host must know specific register addresses, memory offsets, or proprietary command structures. This creates tight coupling between hardware and software, resulting in poor portability and high integration complexity.

PateLink replaces this model with a semantic abstraction layer centered on the **On-Device Abstraction (OSA) Semantic KV-Table**, aka. **OSA Dictionary**. Instead of requiring the host to navigate hardware-specific maps, each PateLink device exposes a structured table of semantic identifiers ("keys") paired with associated values. This transforms device interaction from address-specific communication into semantically discoverable control.

2.2. The OSA Semantic KV-Table (aka. OSA Dictionary)

Every PateLink target device maintains an internal OSA Dictionary composed of compact semantic keys and associated, typed values.

2.2.1. OSA Keys (Semantic Identifiers)

Keys are concise, human-readable character arrays used for device discovery and introspection. To balance human readability with the constraints of memory-limited MCUs and bandwidth-limited buses, keys are intentionally short.

The architecture optimizes both discovery and runtime performance by separating the two:

- **Discovery:** During initialization, the host retrieves the device’s key list once using semantic strings.
- **Execution:** Each key is mapped to a unique index. All subsequent read/write operations occur via this index rather than repeated string parsing.

This design provides the benefits of semantic self-description while maintaining near-register-level operational efficiency. It eliminates the need for external symbol-to-address conversion tables without the overhead of continuous string parsing. Technically, keys are implemented as contiguous arrays of ASCII `uint8_t` characters, ensuring compatibility across both resource-constrained firmware and high-level host environments.

2.2.2. OSA Values (Typed Data Fields)

Each key maps to a value with a fixed data type and a defined semantic role. While keys are immutable identifiers, associated values may be configured as read-only, write-only, or read-write. This allows interaction through meaningful capabilities rather than proprietary memory locations.

2.2.3. OSA Dictionary Illustration

The following represents a simplified view of a device's OSA Dictionary during runtime:

```
{"brightness": 24, "btn0": 1}
```

In this instance, the host interprets the `brightness` key as a scalar value representing current light intensity, and the `btn0` key as a boolean state indicating a button press. The specific data types, precision, and valid operational ranges for these identifiers are not encoded in the value itself, but are defined by the corresponding Device Type Specification (DTS). This separation ensures that while the dictionary provides the *data*, the DTS provides the *context*.

2.3. Device Type Specification (DTS)

A semantic protocol alone cannot prevent fragmentation if device meanings remain inconsistent. To ensure standardization, PateLink introduces the **Device Type Specification (DTS)**, which defines the canonical semantic blueprint for device classes. For each supported device type, the DTS specifies:

- Required keys and definitions.
- Data types and valid ranges.
- Precision requirements and operational constraints.
- State dependencies.

The DTS balances strict interoperability with ecosystem extensibility. It defines a standardized core of mandatory keys for common categories (e.g., motors, sensors, light sources) to ensure baseline cross-vendor compatibility. Manufacturers can extend these profiles with proprietary keys for advanced, application-specific features. Because these extended keys utilize the same semantic mechanism as standard DTS keys, the host interacts with both core and extended functionality through a uniform model. This allows for innovation without breaking the standard or bypassing the semantic layer.

2.3.1. Runtime Type Introspection

While the DTS defines what a key means, it does not always require one fixed internal data format. This allows manufacturers to choose the most efficient data representation for their hardware (such as integer or floating-point) without breaking semantic compatibility.

To support this flexibility, PateLink uses Runtime Type Introspection. During device discovery, the host can query each key's data type from the device itself. This allows the host to automatically adapt to the device's specific implementation while preserving standardized semantic behavior. In practice, this means devices can remain hardware-optimized internally while still behaving consistently within the broader PateLink ecosystem.

2.3.2. Composite Device Architecture

PateLink devices are not limited to a single functional identity. A single physical device may expose multiple DTS-compliant semantic profiles simultaneously. For example, a single system could semantically present as a sensor, a motion controller, and a light source. This compositional model enables modular design and supports scalable automation architectures.

2.3.3. DTS Governance Model

To prevent "semantic drift," DTS governance requires authoritative control over official core specifications. In the canonical PateLink implementation, standardization is centrally curated by the project's architectural authority.

However, to ensure long-term ecosystem resilience, the framework is released under a GPLv3-only open-source license. This allows the community to fork the specification if governance fails to meet evolving industry needs.

3. The Unified Programming Interface

While the OSA Dictionary and Device Type Specification (DTS) establish the communication architecture, a complete control paradigm must also address software interaction. PateLink extends beyond protocol standardization into a hierarchical programming model designed to bridge the gap between embedded hardware and high-level software development.

The objective is to shift developers from **bit-level control** toward **intent-level control** while preserving complete semantic access when required. To achieve this, PateLink defines a two-tiered programming structure: the **Direct API** and the **Driver API**.

3.1. The Direct API: OSA-Level Universal Access

The Direct API is the foundational software interface of the PateLink ecosystem. It operates directly at the level of the OSA Dictionary, providing transparent access to the semantic keys and values exposed by connected devices.

At this layer, software interacts with devices through their discovered OSA structures. The Direct API does not interpret DTS semantics, classify device types, or impose assumptions about device functionality; it simply provides the mechanism to discover, enumerate, read, and write semantic OSA keys and values.

This distinction is fundamental: the Direct API abstracts the **mechanics of communication** but not the **meaning of the data**. Consequently, the Direct API provides:

- **Full Transparency:** Access to both standardized and manufacturer-specific capabilities.
- **Universal Substrate:** A consistent method for interacting with any PateLink-compliant device.
- **Low-Level Control:** Direct access to all exposed semantic keys without abstraction overhead.

Because the Direct API does not encode DTS behavior, developers using this layer must still understand the semantic purpose of keys through DTS documentation or manufacturer specifications. It removes transport-layer fragmentation, but the responsibility for semantic interpretation remains with the user.

3.2. The Driver API: DTS-Derived Semantic Control

Where the Direct API provides universal access, the Driver API provides semantic productivity.

The Driver API is a higher-level layer built upon the Direct API. Instead of exposing raw key-value interactions, it provides predefined functional interfaces whose behavior is implemented according to DTS definitions.

A critical architectural distinction is that the Driver API does not dynamically parse DTS specifications at runtime. Instead, DTS knowledge is deliberately encoded into the driver implementation. The workflow is as follows:

1. **DTS** defines the semantic rules.
2. **Driver implementations** encode those rules into functions.
3. **Developers** utilize the resulting functional interface.

For standard device categories, end-users do not need to manage OSA key structures or manually sequence operations; they interact exclusively with the Driver API.

3.2.1. Conceptual Example

Consider a device type where the DTS specifies that a specific sequence of key writes triggers rotation:

- **Direct API Approach:** A developer must manually execute the sequence (e.g., Write Key A → Write Key B → Write Key C) to achieve clockwise rotation. The developer must understand this underlying logic of the sequence from DTS.
- **Driver API Approach:** The developer calls a single semantic function: `rotate_clockwise()`.

The Driver API is not merely "syntactic sugar"; it is a semantic behavioral layer that transforms protocol mechanics into reusable, intent-based control logic.

3.3. Summary: Balancing Power and Productivity

The Direct and Driver APIs are not competing approaches, but hierarchical layers within a unified architecture. This structure allows developers to choose the level of abstraction that matches their requirements (**Table 1**).

By providing both layers, PateLink ensures that software remains both semantically productive for standard tasks and fundamentally connected to the underlying hardware for specialized requirements.

Table 1. Comparision of Direct and Driver APIs.

	Direct API	**Driver API**
Primary Goal	Transparency & Control	Portability & Productivity
Abstraction Level	OSA-Level (Keys/Values)	DTS-Level (Functions/Intent)
Intended for	Diagnostics, custom extensions, and low-level tuning.	Standardized automation and rapid system integration.

4. Conclusion

Embedded device management remains constrained by fragmentation across hardware, firmware, and software ecosystems. While transport standards are abundant, the absence of unified semantic control imposes heavy integration burdens, limits interoperability, and reinforces vendor lock-in.

PateLink addresses this through two integrated pillars: a semantic communication protocol and a unified programming interface. By using the OSA Dictionary and Device Type Specification (DTS), PateLink replaces register-centric dependency with a standardized semantic abstraction. This shifts the focus from connection-based compatibility to capability-based interoperability.

More than a mere protocol, PateLink is an end-to-end device control paradigm. It unifies how embedded devices are discovered, described, and controlled across the entire hardware-to-software stack. By bridging the gap between transport and semantics, PateLink provides a coherent, extensible, and vendor-independent architecture for the future of automated systems.